Real-Time Multi-Camera Video Analytics System

Đặc tả dự án, kiến trúc, tiêu chí nghiệm thu và lộ trình 5 tuần

Phiên bản	2.0 (chỉnh sửa từ bản 1.0)	Ngày cập nhật	07/11/2026
Tác giả	Tuan	Trạng thái	Bản nháp để rà soát
Loại dự án	Portfolio Computer Vision (MVP)	Thời lượng	5 tuần, không mua thêm phần cứng

01 Mục tiêu và phạm vi

1.1 Bài toán

Xây dựng một hệ thống thị giác máy tính **end-to-end** nhận nhiều luồng video gần thời gian thực, phát hiện và theo dõi người, **đếm lượt đi qua vạch** theo chiều, **đo số người và thời gian lưu trú** trong vùng quan tâm (ROI), rồi cung cấp kết quả qua REST API và dashboard. Ví dụ kịch bản áp dụng: đếm lượt ra/vào ở cửa, đo số người và thời gian chờ tại sảnh chờ thang máy hoặc quầy dịch vụ.

1.2 Mục tiêu đo được

M1. Xử lý đồng thời **tối thiểu 2 luồng RTSP** trên một GPU, mỗi luồng có bộ theo dõi (tracker) độc lập.

M2. Sai số đếm lượt qua vạch $\leq 5\%$ trên bộ clip kiểm thử có gán nhãn (mục 6).

M3. Độ trễ xử lý p95 ≤ 300 **ms** mỗi khung hình ở cấu hình chuẩn (mục 4).

M4. Tự kết nối lại trong ≤ 10 **s** sau khi nguồn phát hoạt động trở lại, không cần khởi động lại dịch vụ.

M5. Toàn bộ hệ thống khởi chạy bằng **một lệnh** `docker compose up`, benchmark tái lập được.

Các ngưỡng trên là mục tiêu đề xuất ban đầu; sẽ chốt lại sau khi có số đo baseline ở Tuần 1.

1.3 Trong phạm vi và ngoài phạm vi MVP

Trong phạm vi	Ngoài phạm vi (để phiên bản sau)
• 1 camera USB thật + 2–4 luồng RTSP mô phỏng từ MP4 • Phát hiện lớp *person* bằng mô hình pretrained • Tracking trong *từng* luồng; đếm theo vạch, ROI đa giác, dwell time • Lưu sự kiện (metadata) vào SQLite; xuất CSV • REST API, dashboard, log, healthcheck, Docker Compose • Benchmark PyTorch vs ONNX Runtime (CUDA)	• Nhận dạng lại người giữa các camera (cross-camera Re-ID) • Nhận dạng khuôn mặt hoặc bất kỳ thông tin định danh nào • Quy đổi quỹ đạo sang mét (cần hiệu chuẩn camera / homography) • Huấn luyện lại hoặc fine-tune mô hình • Xác thực người dùng, phân quyền, triển khai cloud • Lưu trữ video dài hạn

1.4 Phần cứng và vai trò

INPUT
1 camera USB/webcam 2D; file MP4 có quyền sử dụng; FFmpeg phát MP4 thành RTSP mô phỏng.

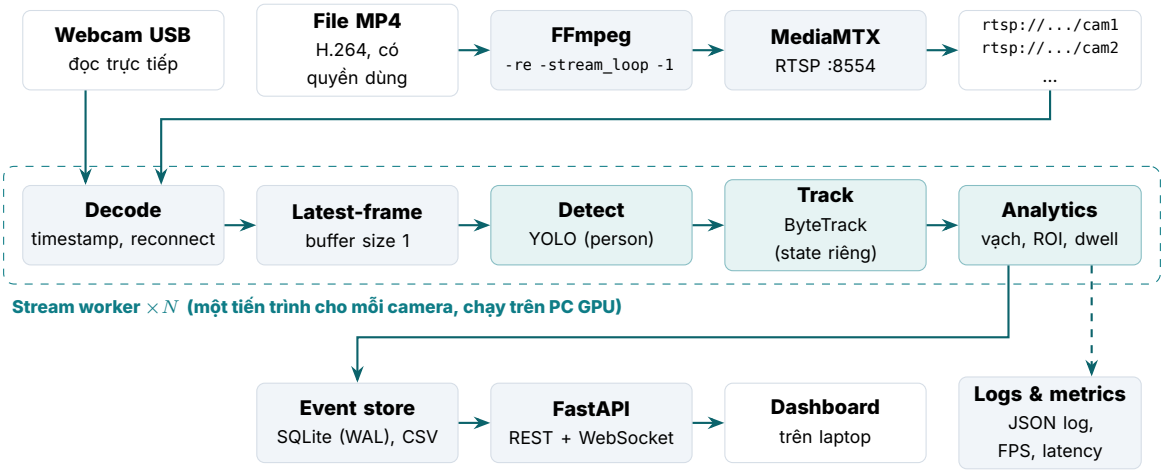
COMPUTE
PC có GPU NVIDIA: MediaMTX, decode, detection, tracking, analytics, cơ sở dữ liệu, API.

CLIENT
Laptop cùng mạng LAN: dashboard, gọi API, chạy kịch bản kiểm thử và bơm lỗi.

Lưu ý Nhiều luồng RTSP phát từ MP4 **không tương đương** nhiều camera vật lý: không có trễ mạng thật, không có lệch đồng hồ, không có góc nhìn chồng lấn. Kết quả benchmark cần ghi rõ điều kiện này. Nếu PC chạy Windows, cần WSL2 và NVIDIA Container Toolkit để container truy cập GPU.

02 Kiến trúc hệ thống

2.1 Sơ đồ tổng quan



2.2 Trách nhiệm từng tầng

Tầng	Trách nhiệm	Công nghệ
1. Ingestion	Phát MP4 thành RTSP theo tốc độ thực; mỗi camera một đường dẫn riêng.	FFmpeg, MediaMTX
2. Decode	Đọc luồng, gắn timestamp, giới hạn buffer, kết nối lại với backoff, báo trạng thái camera.	OpenCV (backend FFmpeg) hoặc PyAV
3. Inference	Phát hiện người; tracker *riêng cho từng luồng*, sinh `track_id` cục bộ theo camera.	Ultralytics YOLO, ByteTrack; ONNX Runtime CUDA
4. Analytics	Đếm qua vạch theo chiều, số người trong ROI, dwell time, thống kê theo phút, lưu quỹ đạo 2D (tọa độ ảnh).	Python, Shapely/NumPy
5. Delivery	Lưu sự kiện, cung cấp API, dashboard, log và metric; đóng gói chạy lặp lại.	SQLite, FastAPI, Streamlit hoặc HTML, Docker Compose

2.3 Quyết định thiết kế quan trọng

- **Cô lập theo camera.** Mỗi camera chạy trong một tiến trình worker riêng; một luồng lỗi không làm dừng luồng khác. Nếu sau này gộp batch detection giữa các luồng để tăng thông lượng, *trạng thái tracker vẫn phải tách riêng* theo `camera_id`.

- **Ưu tiên khung hình mới nhất.** Buffer kích thước 1, ghi đè khung cũ, để độ trễ không tích lũy khi GPU chậm hơn nguồn. Đánh đổi: bỏ khung làm khoảng cách giữa hai khung không đều, ảnh hưởng tracker. Vì vậy phải **đếm và ghi log số khung bị bỏ**, và đặt `frame_rate`/`track_buffer` của ByteTrack theo FPS xử lý thực tế, không theo FPS nguồn.
- **Thời gian.** Mọi phép tính dwell time và latency dùng đồng hồ đơn điệu (`time.monotonic_ns`) tại thời điểm nhận khung; sự kiện lưu thêm thời gian UTC để hiển thị. Không dựa vào PTS của RTSP vì PTS bị đặt lại mỗi lần MP4 lặp.
- **Cấu hình khai báo.** Mỗi camera có file YAML riêng (nguồn, vạch, ROI, ngưỡng); thay đổi cấu hình không cần sửa mã.
- **Ghi dữ liệu.** SQLite ở chế độ WAL với một tiến trình ghi duy nhất nhận sự kiện qua hàng đợi, tránh khóa khi nhiều worker cùng ghi.

03 Đặc tả chức năng

3.1 Yêu cầu chức năng và tiêu chí chấp nhận

Mã	Yêu cầu	Tiêu chí chấp nhận
FR-01	Nhận video từ USB, MP4, RTSP	Cả ba loại nguồn chạy qua cùng một giao diện `VideoSource`.
FR-02	Tự kết nối lại	Backoff 1 s $\rightarrow$ 2 s $\rightarrow$ ... tối đa 30 s; trạng thái `ONLINE / RECONNECTING / OFFLINE` hiển thị qua API.
FR-03	Buffer giới hạn	Bộ nhớ worker ổn định (tăng $< 10\%$) sau 2 giờ chạy liên tục.
FR-04	Phát hiện người	Ngưỡng `conf`, `imgsz`, lớp mục tiêu cấu hình được theo camera.
FR-05	Tracking theo luồng	`track_id` duy nhất trong phạm vi một camera; không chia sẻ ID giữa camera.
FR-06	Đếm qua vạch	Mỗi `track_id` được đếm tối đa một lần cho mỗi chiều; có chiều `IN/OUT`.
FR-07	ROI đa giác, occupancy	Số người trong ROI cập nhật mỗi khung; hỗ trợ nhiều ROI mỗi camera.
FR-08	Dwell time	Tính từ lúc vào đến lúc ra ROI theo timestamp; chịu được che khuất ngắn (mục 3.2).
FR-09	Thống kê theo phút	Bảng tổng hợp: lượt IN/OUT, occupancy trung bình/cực đại, dwell trung bình.
FR-10	REST API	Các endpoint ở mục 3.4; có tài liệu OpenAPI tự sinh.
FR-11	Dashboard	Trạng thái camera, bộ đếm, biểu đồ theo phút, FPS và latency từng luồng.
FR-12	Logging & metric	Log JSON có `camera_id`; metric FPS, latency, dropped frames, GPU/RAM.
FR-13	Đóng gói	`docker compose up` khởi chạy MediaMTX, publisher, worker, API, dashboard.

3.2 Logic analytics

Điểm neo. Mỗi người được biểu diễn bằng điểm giữa cạnh dưới bounding box (gần vị trí chân), ổn định hơn tâm hộp khi người bị che một phần.

Đếm qua vạch. Với vạch AB và điểm neo P_t tại khung t, tính khoảng cách có dấu

$$s_t = \frac{(B - A) \times (P_t - A)}{\|B - A\|}.$$

Một lượt qua được ghi nhận khi s đổi dấu *và* điểm đã đi ra khỏi dải trễ $|s| > \delta$ ở cả hai phía (hysteresis, ví dụ $\delta = 10$ px), đồng thời giao điểm nằm trong đoạn AB. Chiều đi suy ra từ dấu của s trước và sau.

Chống đếm trùng. (1) chỉ đếm track đã tồn tại tối thiểu N_{min} khung (ví dụ 5); (2) mỗi cặp (`track_id`, chiều) chỉ đếm một lần; (3) thời gian chờ (cooldown) giữa hai lượt ngược chiều của cùng track; (4) ghi lại số lần đổi ID (ID switch) để đánh giá ảnh hưởng lên sai số đếm.

Occupancy và dwell time. Một track được coi là *vào* ROI sau k khung liên tiếp có điểm neo nằm trong đa giác (ví dụ $k = 3$) và *ra* khi nằm ngoài, hoặc mất dấu, quá thời gian ân hạn T_{grace} (ví dụ 1,5 s) để chịu được che khuất ngắn.

$$\text{dwell} = t_{\text{exit}} - t_{\text{enter}},$$

trong đó t_{exit} là thời điểm cuối cùng còn thấy track trong ROI, không phải thời điểm hết ân hạn.

3.3 Mô hình dữ liệu sự kiện

```
{
  "event_id":   "c1-000123",
  "camera_id":  "cam1",
  "type":       "line_cross",          // line_cross | zone_enter | zone_exit
  "ts_utc":     "2026-11-07T09:15:32.481Z",
  "track_id":   42,                    // chỉ duy nhất trong cam1
  "line_id":    "door_main",           // hoặc "zone_id" với sự kiện ROI
  "direction":  "IN",                  // IN | OUT (chỉ với line_cross)
  "dwell_s":    null,                  // có giá trị với zone_exit
  "confidence": 0.81
}
```

3.4 REST API

Endpoint	Mô tả
`GET /health`	Trạng thái dịch vụ và từng camera; dùng cho Docker healthcheck.
`GET /cameras`	Danh sách camera, trạng thái, FPS và latency hiện tại.
`GET /events`	Lọc theo `camera_id`, `type`, `from`, `to`; phân trang bằng `limit`, `cursor`.
`GET /stats/counts`	Lượt IN/OUT theo khoảng (`interval=1m`, `5m`, `1h`).
`GET /stats/zones`	Occupancy hiện tại, trung bình, cực đại và dwell time theo ROI.
`WS /ws/events`	(Tùy chọn) đẩy sự kiện thời gian thực cho dashboard.

3.5 Cấu hình camera (ví dụ)

```
camera_id: cam1
source: rtsp://mediamtx:8554/cam1
detector: {model: yolo11n.onnx, imgsz: 640, conf: 0.35, classes: [person]}
tracker:  {type: bytetrack, track_buffer: 30}
lines:
  - {id: door_main, a: [420, 610], b: [980, 600], hysteresis_px: 10}
zones:
  - {id: lobby, polygon: [[300,400],[1100,380],[1180,700],[260,720]],
     enter_frames: 3, grace_s: 1.5}
```

Phát MP4 thành luồng RTSP (yêu cầu video H.264 để dùng `-c copy`):

```
ffmpeg -re -stream_loop -1 -i lobby.mp4 -c copy -f rtsp rtsp://localhost:8554/cam1
```

04 Yêu cầu phi chức năng

Cấu hình chuẩn để so sánh: 2 luồng 1280×720 @ 25 FPS, `imgsz=640`, mô hình cỡ *nano*, FP16 nếu GPU hỗ trợ. Thông số phần cứng (GPU, VRAM, CPU, RAM, driver, phiên bản CUDA) phải ghi trong mọi báo cáo benchmark.

Chỉ số	Định nghĩa đo	Mục tiêu MVP
Thông lượng	Số khung xử lý xong mỗi giây, mỗi luồng	≥ 15 FPS/luồng
Độ trễ xử lý	Từ lúc worker nhận khung đến lúc sự kiện được phát ra; báo cáo p50/p95/p99	p95 ≤ 300 ms
Dropped frames	Tỉ lệ khung bị ghi đè trong buffer trên tổng số khung nhận	$\leq 5\%$ ở cấu hình chuẩn
Phục hồi	Từ lúc nguồn hoạt động lại đến lúc worker `ONLINE`	≤ 10 s
Ổn định	Chạy liên tục (soak test)	2 giờ, không crash, RAM tăng $< 10\%$
Tài nguyên	GPU util, VRAM, CPU, RAM theo số luồng	Ghi nhận, không đặt ngưỡng

Độ trễ end-to-end "glass-to-glass" (từ camera đến màn hình) là chỉ số bổ sung, đo bằng cách quay đồng hồ ms và so với khung hiển thị.

05 Kế hoạch triển khai 5 tuần

Nguyên tắc: **đo ngay từ Tuần 1**. Mã đo FPS/latency và bộ clip kiểm thử phải có trước khi tối ưu.

Tuần	Chủ đề	Công việc chính	Đầu ra nghiệm thu
1	CV baseline & đo lường	YOLO + ByteTrack trên USB/MP4; xuất video có bounding box, ID, quỹ đạo. Gắn timestamp và log latency từng tầng. Chọn 3–5 clip kiểm thử (2–5 phút), gán nhãn số lượt qua vạch và dwell time bằng CVAT.	Demo 1 camera; bảng số đo baseline; bộ ground truth.
2	Analytics	Vạch có hysteresis, ROI đa giác, dwell time với ân hạn, thống kê theo phút. Unit test bằng quỹ đạo dựng sẵn (đi qua, đứng trên vạch, quay đầu, đổi ID, mất dấu ngắn).	Bộ test `pytest` xanh; sai số đếm trên clip kiểm thử.
3	Multi-stream & API	FFmpeg phát 2–4 luồng RTSP qua MediaMTX; worker theo camera, reconnect, latest-frame buffer; SQLite; FastAPI; Docker Compose có GPU.	API chạy đa luồng, `/health` hoạt động; `docker compose up` một lệnh.
4	Dashboard & độ bền	Dashboard trên laptop. Bơm lỗi: tắt MediaMTX, dừng publisher, ngắt mạng; soak test 2 giờ.	Báo cáo kiểm thử lỗi; dashboard hoàn chỉnh.
5	Tối ưu & đóng gói	So sánh PyTorch với ONNX Runtime CUDA (FP32/FP16) theo ma trận benchmark; (mở rộng) TensorRT EP. README, sơ đồ kiến trúc, video demo 1–2 phút.	Báo cáo benchmark tái lập; repo hoàn chỉnh; video demo.

Dự phòng: nếu chậm tiến độ, phần TensorRT và WebSocket là mục được cắt trước; không cắt kiểm thử và benchmark.

06 Đánh giá và kiểm thử

6.1 Dữ liệu đánh giá

- **Tracking:** tập MOT17 (MOTChallenge) đánh giá bằng TrackEval để có số HOTA/IDF1 so sánh được với công bố.
- **Đếm và dwell time:** 3–5 clip tự quay hoặc có giấy phép, gán nhãn thủ công số lượt IN/OUT và thời điểm vào/ra ROI.
- **Unit test:** quỹ đạo tổng hợp với kết quả biết trước, không phụ thuộc mô hình.

6.2 Chỉ số

Chỉ số	Ý nghĩa	Công cụ
Chất lượng CV		
mAP@0.5 (person)	Chất lượng phát hiện; *tùy chọn* vì dùng mô hình pretrained, chỉ đo khi có nhãn bounding box	Ultralytics val / pycocotools
HOTA, IDF1, ID switch	Chất lượng tracking và độ ổn định ID	TrackEval
Sai số đếm	$\lvert\hat{n} - n\rvert / n$ theo từng vạch và chiều	Script so với ground truth
MAE dwell time	Trung bình $\lvert\hat{d} - d\rvert$ theo giây cho các lượt khớp	Script so với ground truth
Hệ thống		
FPS, latency p50/p95/p99	Theo định nghĩa ở mục 4	Metric nội bộ
Dropped frames	Tỉ lệ khung bị bỏ	Metric nội bộ
GPU/VRAM/CPU/RAM	Tài nguyên theo số luồng	`nvidia-smi`, psutil

6.3 Giao thức benchmark

- Ma trận: mô hình {n, s} × `imgsz` {480, 640} × backend {PyTorch, ORT CUDA FP32, ORT CUDA FP16} × số luồng {1, 2, 4}.
- Mỗi cấu hình: bỏ 100 khung khởi động, đo 5 phút, lặp 3 lần, báo cáo trung vị.
- Kết quả lưu CSV kèm commit hash, cấu hình và thông số phần cứng; script `make bench` tái tạo được toàn bộ bảng.
- Kiểm tra độ chính xác sau khi chuyển ONNX/FP16: sai số đếm không được kém hơn bản PyTorch quá 1 điểm phần trăm.

6.4 Kiểm thử bơm lỗi

Kịch bản	Hành vi mong đợi
Dừng publisher của `cam1` 30 s	`cam1` chuyển `RECONNECTING`, `cam2` không bị ảnh hưởng; tự `ONLINE` trong ≤ 10 s khi phát lại.
Khởi động lại container MediaMTX	Tất cả worker tự kết nối lại, không cần khởi động lại worker.
Nguồn phát nhanh hơn khả năng xử lý	Latency không tăng dần theo thời gian; dropped frames tăng và được ghi log.
Dữ liệu cấu hình sai (ROI không hợp lệ)	Worker báo lỗi rõ ràng khi khởi động, không crash các worker khác.

07 Definition of Done

- ❑ Tối thiểu 2 luồng RTSP mô phỏng + 1 camera USB chạy đồng thời qua `docker compose up`.
- ❑ Đạt các mục tiêu M1–M5, hoặc có giải thích bằng số đo nếu chưa đạt.
- ❑ Tự phục hồi khi mất luồng; toàn bộ kịch bản bơm lỗi ở mục 6.4 đạt.
- ❑ Unit test analytics xanh; báo cáo sai số đếm, MAE dwell time, HOTA/IDF1.
- ❑ Báo cáo benchmark tái lập (CSV + script + thông số phần cứng).
- ❑ Log JSON, endpoint `/health`, dashboard hiển thị trạng thái và metric.
- ❑ README (cài đặt, cấu hình, giới hạn), sơ đồ kiến trúc, video demo 1–2 phút.

08 Rủi ro và phương án giảm thiểu

Rủi ro	Giảm thiểu
GPU không đủ cho 4 luồng	Giảm `imgsz`, dùng mô hình nano, FP16, xử lý cách khung (frame stride) có ghi log; báo cáo giới hạn thực tế.
ID switch gây đếm trùng hoặc sót	Hysteresis, tuổi track tối thiểu, cooldown; đo ID switch và tương quan với sai số đếm.
PTS bị đặt lại khi MP4 lặp	Dùng đồng hồ đơn điệu tại thời điểm nhận khung cho mọi phép tính thời gian.
Docker không truy cập được GPU	Cài NVIDIA Container Toolkit (Windows: qua WSL2); kiểm tra bằng `nvidia-smi` trong container ngay Tuần 3.
Che khuất, đám đông dày	Ân hạn T_{grace} cho ROI; chọn góc camera từ trên cao; ghi rõ giới hạn trong README.
Phiên bản CUDA/cuDNN không khớp ONNX Runtime	Cố định phiên bản trong Dockerfile theo bảng tương thích chính thức.

09 Quyền riêng tư và giấy phép

- **Dữ liệu video:** chỉ dùng video tự quay có sự đồng ý hoặc video có giấy phép sử dụng rõ ràng; ghi nguồn trong README.
- **Không lưu danh tính:** hệ thống chỉ lưu metadata (ID cục bộ, tọa độ, thời gian); không lưu khuôn mặt, không lưu video gốc lâu dài. Video demo công khai nên làm mờ khuôn mặt.
- **Triển khai thực tế:** nếu đặt camera tại tòa nhà hoặc nơi làm việc, cần thông báo khu vực có camera và tuân thủ quy định hiện hành về bảo vệ dữ liệu cá nhân tại Việt Nam.
- **Giấy phép mô hình:** Ultralytics YOLO phát hành theo AGPL-3.0. Dùng cho portfolio mã nguồn mở là phù hợp; nếu dùng trong sản phẩm thương mại đóng cần giấy phép Enterprise hoặc thay bằng mô hình có giấy phép thoáng hơn (ví dụ YOLOX, Apache-2.0).

10 Cấu trúc repository đề xuất

```
mcva/
  configs/            cam1.yaml, cam2.yaml, bench.yaml
  src/mcva/
    ingest/           video_source.py, reconnect.py
    inference/        detector.py (torch | onnx), tracker.py
    analytics/        line_counter.py, zone.py, stats.py
    storage/          writer.py (SQLite WAL), schema.sql
    api/              main.py (FastAPI), routes/
    metrics/          timing.py, exporter.py
  dashboard/          app.py
  tests/              test_line_counter.py, test_zone.py, fixtures/
  bench/              run_bench.py, results/*.csv
  docker/             Dockerfile.worker, Dockerfile.api
  docker-compose.yml  README.md  Makefile
```

11 Tài liệu tham khảo (ưu tiên theo thứ tự đọc)

[1] Ultralytics — Multi-Object Tracking: `track()`, `persist`, ByteTrack, BoT-SORT. https://docs.ultralytics.com/modes/track/

[2] MediaMTX — RTSP/WebRTC server. https://github.com/bluenviron/mediamtx; FFmpeg documentation. https://ffmpeg.org/ffmpeg.html

[3] Y. Zhang et al., *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*, ECCV 2022.

[4] TrackEval — HOTA, IDF1, CLEAR MOT. https://github.com/JonathonLuiten/TrackEval

[5] J. Luiten et al., *HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking*, IJCV 2021.

[6] E. Ristani et al., *Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking* (IDF1), ECCV Workshops 2016.

[7] MOTChallenge — MOT17 benchmark. https://motchallenge.net

[8] ONNX Runtime — CUDA Execution Provider. https://onnxruntime.ai/docs/execution-providers/CUDA-ExecutionProvider.html

[9] FastAPI — First Steps. https://fastapi.tiangolo.com/tutorial/first-steps/

[10] Docker Compose — Quickstart. https://docs.docker.com/compose/gettingstarted/

[11] NVIDIA Container Toolkit — Installation Guide. https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html

[12] CVAT — công cụ gán nhãn video. https://github.com/cvat-ai/cvat